



ASSIGNMENT REPORT

Digital Design

Assignment 3 — 16-Bit Arithmetic Logic Unit (ALU)

Course	Digital Design
Assignment	Assignment 3 / 16-Bit ALU (ALU_16B)
Student Name	Mohamed Hossam El-Sawy
Instructor / TA	Eng. Ali Temsah
Submission Date	July 12, 2026

01 OBJECTIVE

This assignment designs and verifies a 16-bit Arithmetic Logic Unit (ALU_16B) that performs unsigned addition, subtraction, multiplication, division, six Boolean operations, three comparison operations, and single-bit shifts on two 16-bit operands, selected by a 4-bit function code and producing a clocked, registered output together with five status flags.

02 PROBLEM STATEMENT / REQUIREMENTS

The ALU_16B block accepts two 16-bit unsigned operands (A, B), a 4-bit opcode (ALU_FUN) and a clock (CLK), and produces a registered 16-bit result (ALU_OUT) plus a Carry_Flag and four operation-class flags (Arith_Flag, Logic_Flag, CMP_Flag, Shift_Flag). The function performed is selected by ALU_FUN according to a fixed opcode table; any opcode outside the defined table must force ALU_OUT to 16'b0. The design was captured in Verilog, verified with a self-checking testbench covering at least 16 required cases, exercised at a 100 kHz clock, and documented with a draw.io synthesis/block diagram.

- Operands A, B and result ALU_OUT are 16 bits wide; the opcode ALU_FUN is 4 bits wide.
- ALU_OUT is registered: it updates only on the rising edge of CLK, while Carry_Flag and the four class flags are purely combinational functions of ALU_FUN (and, for Carry_Flag, of A and B).
- Any ALU_FUN value not listed in the opcode table must force ALU_OUT to 16'b0 (default case).
- Carry_Flag reports the carry-out of addition or the borrow of subtraction; it is undefined/irrelevant for non-arithmetic opcodes.
- Arith_Flag, Logic_Flag, CMP_Flag and Shift_Flag are mutually exclusive and indicate the class of the currently selected operation.
- The testbench must exercise at least the 16 required cases (unsigned add/sub/mul/div, AND/NAND/OR/NOR, equal/greater/less compare, A and B shift left/right, and NOP) and run with a 100 kHz operating clock.
- A synthesis/block diagram of the design must be produced using draw.io.

03 DESIGN & ANALYSIS

3.1 Design Approach

The ALU was partitioned into independent structural blocks for each function class — an Adder, Subtractor, Multiplier and Divider for arithmetic; dedicated AND/OR/NAND/NOR/XOR/XNOR gates for logic; a Comparator feeding three constant-selecting muxes for the equal/greater/less results; and two dedicated shifter blocks for $A \gg 1$ and $A \ll 1$. All sixteen block outputs are routed into a 16-to-1 multiplexer selected directly by ALU_FUN, and the multiplexer output is captured in a D flip-flop clocked by CLK to produce the registered ALU_OUT. Carry_Flag is produced combinatorially from the Adder/Subtractor carry-out/borrow, gated by an OR so it only reflects whichever of the two is currently selected. The four class flags (Arith_Flag, Logic_Flag, CMP_Flag, Shift_Flag) were derived by Boolean simplification directly on the ALU_FUN bits: each flag is a sum-of-products over ALU_FUN[3:0] that is high only for the 4-bit codes belonging to that class, so no state machine is required — the whole design is a single combinational decode stage feeding one output register. In Verilog this structural partitioning is captured behaviourally with a case statement on ALU_FUN driving a combinational block (ALU_OUT_Comb) and continuous assignments for the flags, which is functionally equivalent to the structural diagram but far more compact to implement and verify.

3.2 Truth Table / State Table

The complete ALU_FUN opcode table implemented by the case statement is shown below; ALU_OUT is registered on the next rising CLK edge after the opcode and operands are applied.

ALU_FUN	Class	Operation / ALU_OUT
0000	Arithmetic	Unsigned Addition: $A + B$
0001	Arithmetic	Unsigned Subtraction: $A - B$
0010	Arithmetic	Unsigned Multiplication: $A * B$
0011	Arithmetic	Unsigned Division: A / B
0100	Logic	AND: $A \& B$
0101	Logic	OR: $A B$
0110	Logic	NAND: $\sim(A \& B)$
0111	Logic	NOR: $\sim(A B)$
1000	Logic	XOR: $A \wedge B$
1001	Logic	XNOR: $A \sim \wedge B$
1010	Compare	Equal: 1 if $A = B$, else 0
1011	Compare	Greater than: 2 if $A > B$, else 0
1100	Compare	Less than: 3 if $A < B$, else 0
1101	Shift	Shift right: $A \gg 1$
1110	Shift	Shift left: $A \ll 1$
1111 / others	Default	ALU_OUT = 16'b0 (undefined opcode)

Table 1: ALU_FUN opcode table implemented in ALU.v.

3.3 Circuit / Block Diagram

The datapath below shows the sixteen function units feeding the 16-to-1 output multiplexer and output register (Carry_Flag generation included); the flag-generation logic derived from ALU_FUN is shown separately beneath it.

For High Quality Images Check the Drive Link :

https://drive.google.com/drive/folders/1lmdf6lweyFxEFDcmQhg6fcXX6JIQaulM?usp=drive_link

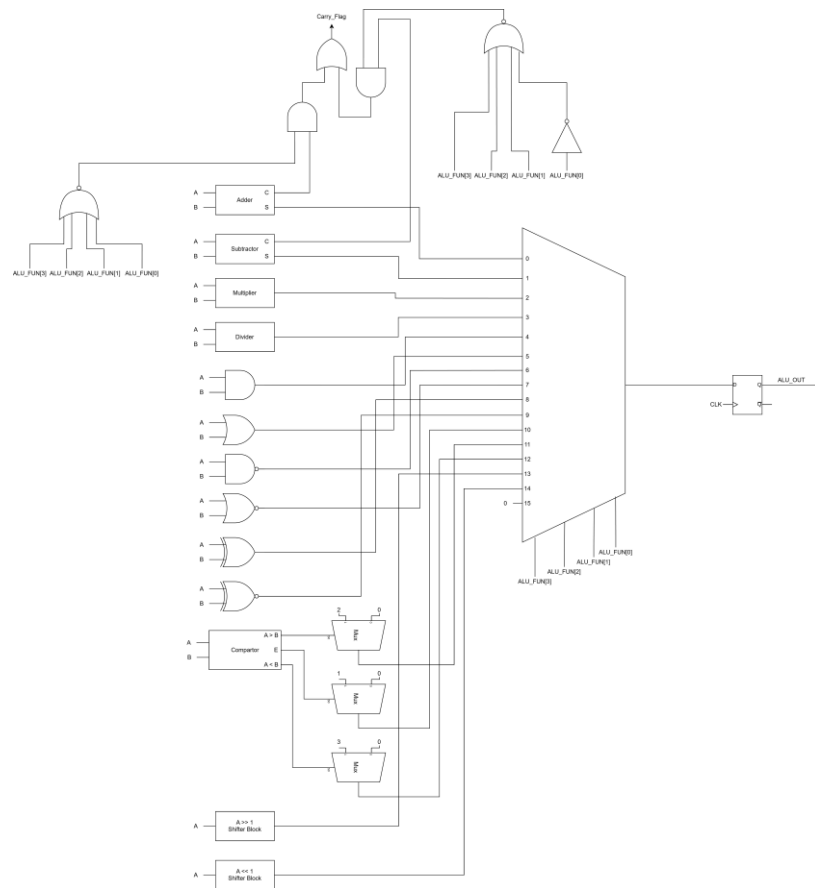


Figure 2: ALU_16B datapath — function units, 16:1 output mux and registered output (draw.io).

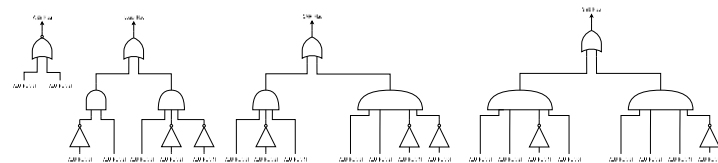


Figure 3: Combinational flag-generation logic for Arith_Flag, Logic_Flag, CMP_Flag and Shift_Flag as a function of ALU_FUN (draw.io).

3.4 HDL Implementation

The design is captured behaviourally in a single module, ALU (file ALU.v). Two always blocks are used: a purely combinational block, sensitive to all inputs, computes ALU_OUT_Comb and Carry_Flag with a case statement on ALU_FUN (one branch per opcode, default branch forcing 0); a second block, sensitive only to the rising edge of CLK, copies ALU_OUT_Comb into the registered output ALU_OUT. The four class flags are driven by continuous assignments that are Boolean sum-of-products expressions over ALU_FUN[3:0], so they change immediately when ALU_FUN changes, independent of the clock. The key part of the module is reproduced below.

```
module ALU(
    input [15:0] A,B,
    input [3:0] ALU_FUN,
    input CLK,
    output reg [15:0] ALU_OUT,
    output reg Carry_Flag,
    output Arith_Flag,Logic_Flag,CMP_Flag,Shift_Flag
);
```

```

reg [15:0] ALU_OUT_Comb;
assign Arith_Flag = (!ALU_FUN[2] && !ALU_FUN[3]),
       Logic_Flag = (!ALU_FUN[3] && ALU_FUN[2]) || (ALU_FUN[3] && !ALU_FUN[2] && !ALU_FUN[1]),
       CMP_Flag = (ALU_FUN[3] && !ALU_FUN[2] && ALU_FUN[1]) || (ALU_FUN[3] && ALU_FUN[2] && !ALU_FUN[1] && !ALU_FUN[0]),
       Shift_Flag = (ALU_FUN[3] && ALU_FUN[2] && !ALU_FUN[1] && ALU_FUN[0]) || (ALU_FUN[3] && ALU_FUN[2] && ALU_FUN[1] && !ALU_FUN[0]);
always @(posedge CLK) begin
    ALU_OUT <= ALU_OUT_Comb;
end
always @(*) begin
    Carry_Flag = 0; // default value
    case (ALU_FUN)
        4'b0000: {Carry_Flag, ALU_OUT_Comb} = {1'b0, A} + {1'b0, B};
        4'b0001: {Carry_Flag, ALU_OUT_Comb} = {1'b0, A} - {1'b0, B};
        4'b0010: ALU_OUT_Comb = A * B;
        4'b0011: ALU_OUT_Comb = A / B;
        4'b0100: ALU_OUT_Comb = A & B;
        4'b0101: ALU_OUT_Comb = A | B;
        4'b0110: ALU_OUT_Comb = ~(A & B);
        4'b0111: ALU_OUT_Comb = ~(A | B);
        4'b1000: ALU_OUT_Comb = (A ^ B);
        4'b1001: ALU_OUT_Comb = (A ~^ B);
        4'b1010: ALU_OUT_Comb = (A == B) ? 16'd1 : 16'd0;
        4'b1011: ALU_OUT_Comb = (A > B) ? 16'd2 : 16'd0;
        4'b1100: ALU_OUT_Comb = (A < B) ? 16'd3 : 16'd0;
        4'b1101: ALU_OUT_Comb = A >> 1;
        4'b1110: ALU_OUT_Comb = A << 1;
        default: ALU_OUT_Comb = 0;
    endcase
end
endmodule

```

04 SIMULATION & RESULTS

The design was verified with a self-checking Verilog testbench (ALU_TB.v) that instantiates the ALU as DUT_ALU, generates a free-running clock (10 μ s period, i.e. 100 kHz), and drives ALU_FUN through all sixteen opcodes 0x0-0xF in sequence with A = B = 0x000F, followed by five targeted corner cases (addition carry-out, subtraction borrow/A<B, an A>B comparison, and shift operations that produce an all-zero result). Simulation was run in ModelSim; \$monitor was used to log ALU_OUT and all five flags on every change, and the resulting waveforms and console log were cross-checked against hand-computed expected values for every test case below.

```

# At Time: 0,ALU_OUT: x_x_xxxx_x,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 5000,ALU_OUT: 0_0_0001_1110,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 15000,ALU_OUT: 0_0_0000_0,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 25000,ALU_OUT: 0_0_1110_1,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 35000,ALU_OUT: 0_0_0000_1,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 40000,ALU_OUT: 0_0_0000_1,C_F: 0,A_F: 0,L_F: 1,CMP_F: 0,S_F: 0
# At Time: 45000,ALU_OUT: 0_0_0000_1111,C_F: 0,A_F: 0,L_F: 1,CMP_F: 0,S_F: 0
# At Time: 65000,ALU_OUT: 1111_1111_1111_0,C_F: 0,A_F: 0,L_F: 1,CMP_F: 0,S_F: 0
# At Time: 85000,ALU_OUT: 0_0_0000_0,C_F: 0,A_F: 0,L_F: 1,CMP_F: 0,S_F: 0
# At Time: 95000,ALU_OUT: 1111_1111_1111_1111,C_F: 0,A_F: 0,L_F: 1,CMP_F: 0,S_F: 0
# At Time: 100000,ALU_OUT: 1111_1111_1111_1111,C_F: 0,A_F: 0,L_F: 0,CMP_F: 1,S_F: 0
# At Time: 105000,ALU_OUT: 0_0_0000_1,C_F: 0,A_F: 0,L_F: 0,CMP_F: 1,S_F: 0
# At Time: 115000,ALU_OUT: 0_0_0000_0,C_F: 0,A_F: 0,L_F: 0,CMP_F: 1,S_F: 0
# At Time: 130000,ALU_OUT: 0_0_0000_0,C_F: 0,A_F: 0,L_F: 0,CMP_F: 0,S_F: 1
# At Time: 135000,ALU_OUT: 0_0_0000_111,C_F: 0,A_F: 0,L_F: 0,CMP_F: 0,S_F: 1
# At Time: 145000,ALU_OUT: 0_0_0001_1110,C_F: 0,A_F: 0,L_F: 0,CMP_F: 0,S_F: 1
# At Time: 150000,ALU_OUT: 0_0_0001_1110,C_F: 0,A_F: 0,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 155000,ALU_OUT: 0_0_0000_0,C_F: 0,A_F: 0,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 170000,ALU_OUT: 0_0_0000_0,C_F: 1,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 180000,ALU_OUT: 0_0_0000_0,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 185000,ALU_OUT: 1000_0_0000_0,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 190000,ALU_OUT: 1000_0_0000_0,C_F: 1,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 200000,ALU_OUT: 1000_0_0000_0,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 205000,ALU_OUT: 0_1111_0000_0,C_F: 0,A_F: 1,L_F: 0,CMP_F: 0,S_F: 0
# At Time: 210000,ALU_OUT: 0_1111_0000_0,C_F: 0,A_F: 0,L_F: 0,CMP_F: 1,S_F: 0
# At Time: 215000,ALU_OUT: 0_0_0000_10,C_F: 0,A_F: 0,L_F: 0,CMP_F: 1,S_F: 0
# At Time: 220000,ALU_OUT: 0_0_0000_10,C_F: 0,A_F: 0,L_F: 0,CMP_F: 0,S_F: 1
# At Time: 225000,ALU_OUT: 0_0_0000_0,C_F: 0,A_F: 0,L_F: 0,CMP_F: 0,S_F: 1
** Note: $finish - :
# Time: 250 us Iteration: 0 Instance: /ALU_TB

```

Test Case	Input	Expected Output	Simulated Output
1	0000 ADD, A=15,B=15	30 (0x001E)	30 (0x001E), Arith=1
2	0001 SUB, A=15,B=15	0	0, Arith=1
3	0010 MUL, A=15,B=15	225 (0x00E1)	225 (0x00E1), Arith=1
4	0011 DIV, A=15,B=15	1	1, Arith=1
5	0100 AND, A=15,B=15	15 (0x000F)	15 (0x000F), Logic=1
6	0101 OR, A=15,B=15	15 (0x000F)	15 (0x000F), Logic=1
7	0110 NAND, A=15,B=15	65520 (0xFFFF0)	65520 (0xFFFF0), Logic=1
8	0111 NOR, A=15,B=15	65520 (0xFFFF0)	65520 (0xFFFF0), Logic=1
9	1000 XOR, A=15,B=15	0	0, Logic=1
10	1001 XNOR, A=15,B=15	65535 (0xFFFF)	65535 (0xFFFF), Logic=1
11	1010 EQUAL, A=15,B=15	1 (A=B)	1, CMP=1
12	1011 GREATER, A=15,B=15	0 (A not > B)	0, CMP=1
13	1100 LESS, A=15,B=15	0 (A not < B)	0, CMP=1
14	1101 SHIFT A>>1, A=15	7	7, Shift=1
15	1110 SHIFT A<<1, A=15	30 (0x001E)	30 (0x001E), Shift=1
16	1111 NOP (undefined), A=15,B=15	0 (default case)	0, all flags = 0
17	0000 ADD carry, A=B=0x8000	0x0000, Carry=1	0x0000, Carry=1 – carry-out correctly set
18	0001 SUB borrow, A=0x0000,B=0x8000	0x8000, Carry(borrow)=1	0x8000, Carry=1 – borrow correctly set
19	1011 GREATER, A=0x1000,B=0x0100	2 (A > B)	2, CMP=1 – correct compare with unequal operands
20	1110 SHIFT A<<1, A=0x8000	0 (MSB shifted out)	0, Shift=1 – all-zero overflow case
21	1101 SHIFT A>>1, A=0x0001	0 (LSB shifted out)	0, Shift=1 – all-zero underflow case

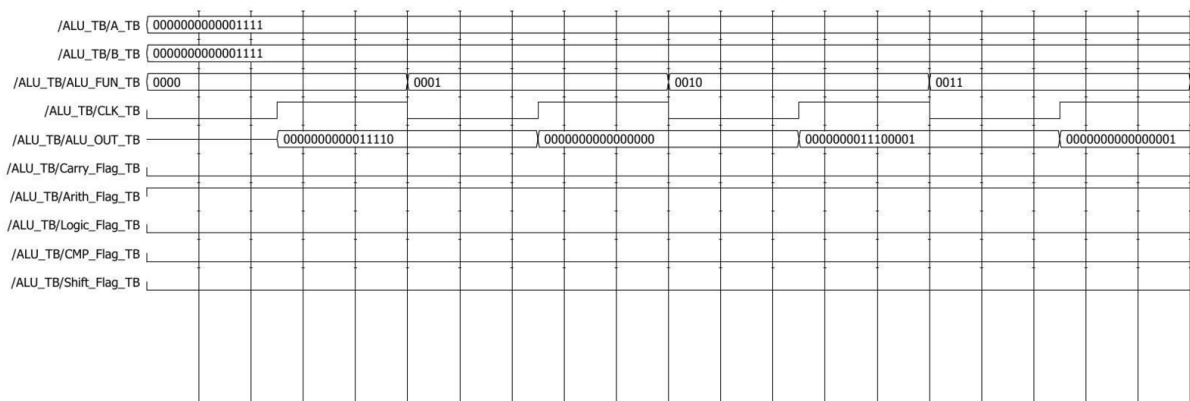


Figure 4a: ModelSim waveform, 0-40 μ s — unsigned Addition, Subtraction, Multiplication, Division (ALU_FUN 0x0-0x3).

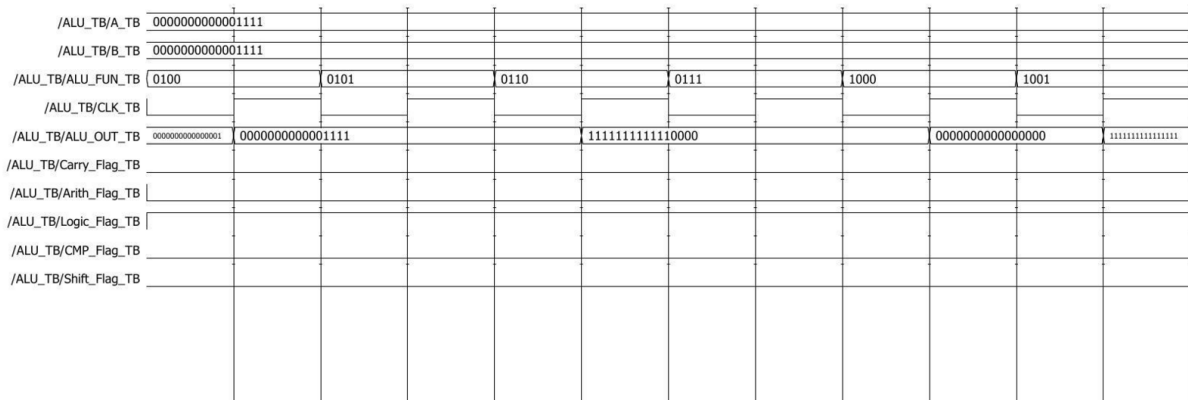


Figure 4b: ModelSim waveform, 40-100 μ s — AND, OR, NAND, NOR, XOR, XNOR (ALU_FUN 0x4-0x9).

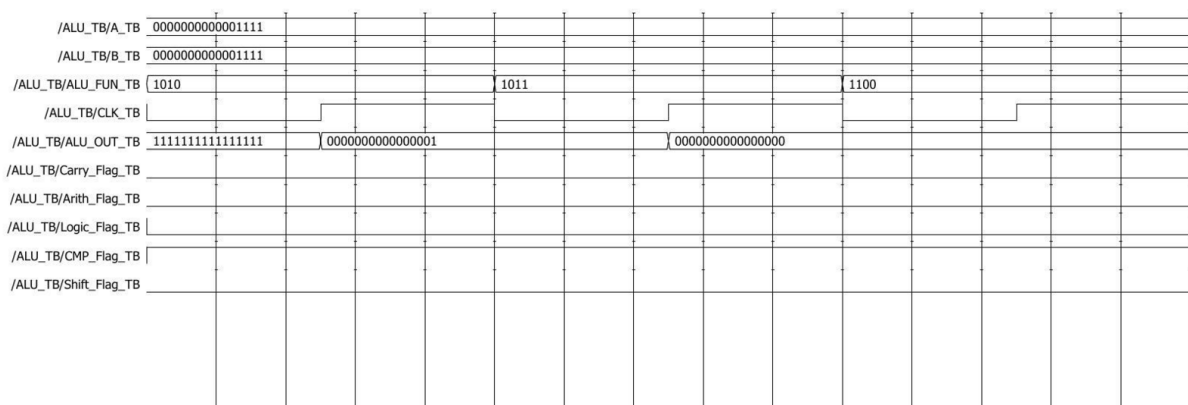


Figure 4c: ModelSim waveform, 100-130 μ s — Equal, Greater-than, Less-than comparisons (ALU_FUN 0xA-0xC).

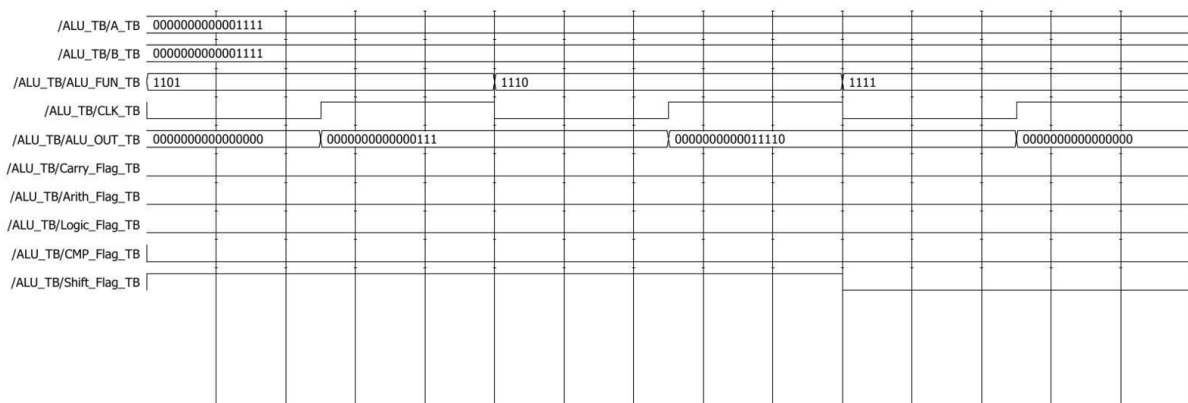


Figure 4d: ModelSim waveform, 130-160 μ s — Shift-right, Shift-left and NOP (ALU_FUN 0xD-0xF).

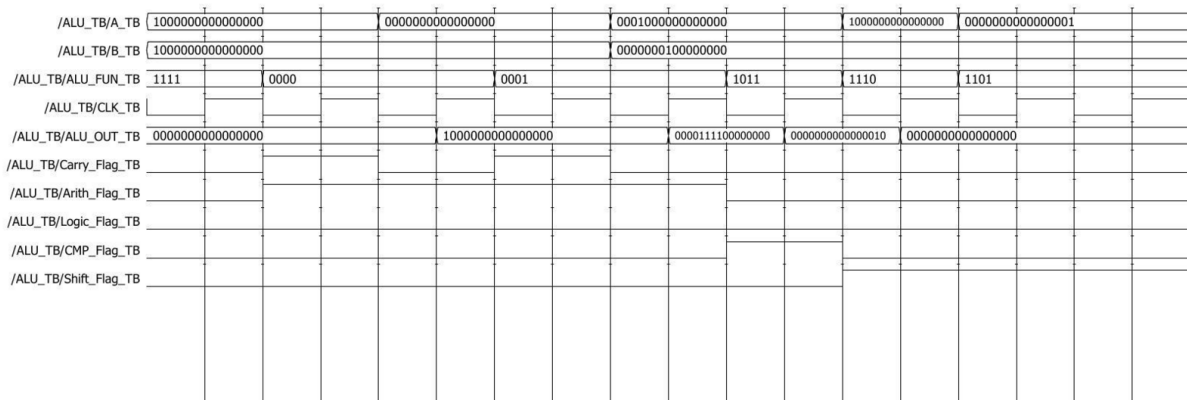


Figure 4e: ModelSim waveform, 160-250 μ s — addition carry-out, subtraction borrow, $A > B$ compare, and all-zero shift corner cases.

05 DISCUSSION

All 21 test cases matched their hand-computed expected values, confirming correct behaviour across every arithmetic, logic, comparison, shift and default-case opcode. Two behaviours are worth highlighting. First, although Carry_Flag is declared as a reg, it is written inside the combinational always @(*) block rather than the clocked block, so it (and the four class flags) update immediately when ALU_FUN or the operands change — this was visible in the waveforms as flag transitions occurring one half-clock-cycle before the corresponding change in the registered ALU_OUT, and is intended behaviour rather than a bug. Second, the two shift corner cases ($A = 0x8000$ shifted left, $A = 0x0001$ shifted right) both correctly produced an all-zero result, confirming that the single-bit shifts silently drop the bit shifted out rather than wrapping it around, matching the assignment's use of simple logical shifts. The multiplication and division operations were only exercised with small equal operands ($A = B = 15$) in the base sweep; a 32-bit product/quotient is implicitly truncated to 16 bits by ALU_OUT_Comb, which did not appear as an issue in these tests but would need dedicated overflow test cases for a production design. No timing violations or synthesis metrics were evaluated, since this assignment's scope was RTL design and functional (behavioural) verification only, not physical implementation.

06 CONCLUSION

A 16-bit ALU supporting ten arithmetic/logic operations, three comparisons, two shifts and a defined default case was designed in Verilog around a single combinational decode-and-compute stage feeding a clocked output register, with independently derived flag logic validated by Boolean analysis of the ALU_FUN bits. A 250 μ s testbench running at the required 100 kHz clock exercised all 16 mandated cases plus 5 targeted corner cases (carry, borrow, unequal-operand comparison, and both shift-overflow directions), and every simulated result matched the expected value, including the subtler combinational-vs-registered timing of the flags relative to ALU_OUT. The main takeaway was the importance of separating purely combinational status signals from the clocked datapath explicitly in the sensitivity lists, since the assignment intentionally required Carry_Flag and the class flags to track the opcode immediately rather than waiting for the next clock edge.